# CS3400: Data Structures and Algorithms

## Mini Project: Intelligent Campus Resource and Activity Management System

### Project Type

Group Mini Project

### Group Size

**Three (3) students per group**

### Programming Language

**C++**

---

# 1. Project Overview

Universities manage a wide range of resources, including lecture rooms, laboratories, equipment, student activities, events, and other facilities. Managing and accessing this information efficiently requires appropriate data structures and algorithms.

For this mini project, each group will design and implement an **Intelligent Campus Resource and Activity Management System** using C++.

The system will manage a collection of campus-related resources and provide efficient mechanisms for:

- Storing and retrieving information.
- Managing resource availability.
- Processing requests.
- Organising activities according to priority.
- Searching and sorting information.
- Maintaining relationships between resources.
- Analysing and navigating connected campus locations.

The project is deliberately designed as a **progressive challenge**. Students will begin with relatively simple data management tasks and gradually introduce more advanced data structures and algorithms as the project develops.

The emphasis of this project is not only on producing a working application, but also on demonstrating an understanding of **why particular data structures and algorithms are appropriate for particular problems**.

---

# 2. Project Scenario

The National University of Lesotho is planning to develop a simplified system for managing campus resources and activities.

The university has many different types of resources and locations. These may include:

- Lecture rooms
- Laboratories
- Computer laboratories
- Offices
- Libraries
- Meeting rooms
- Sports facilities
- Student facilities
- Equipment

Different users may need to search for resources, request access to them, reserve them, or obtain information about their availability.

At the same time, certain activities and requests may be more urgent or important than others. The system must therefore be capable of managing requests according to appropriate priorities.

Campus locations are also interconnected. For example, a student may need to determine how to move from one building to another using available pathways.

Your group has been asked to develop a **prototype system** that demonstrates how appropriate data structures and algorithms can be used to solve these problems efficiently.

---

# 3. Overall Project Goal

The goal of this project is to design and implement a C++ application that demonstrates the practical application of major data structures and algorithms covered in CS3400.

The final system should demonstrate the use of:

- Arrays and/or dynamic arrays
- Linked lists
- Stacks
- Queues
- Trees
- Priority queues or heaps
- Hash tables
- Sorting algorithms
- Searching algorithms
- Graphs
- Algorithm analysis

---

# 4. Learning Objectives

Upon successful completion of this project, students should be able to:

1. Design appropriate abstract data types for a real-world problem.
2. Select suitable data structures based on application requirements.
3. Implement and use dynamic data structures in C++.
4. Apply searching and sorting algorithms.
5. Organise hierarchical information using trees.
6. Manage prioritised requests using heaps or priority queues.
7. Implement efficient information retrieval using hashing.
8. Model relationships using graphs.
9. Analyse the time and space complexity of selected operations.
10. Work collaboratively in designing and implementing a non-trivial software system.

---

# 5. Team Structure

Each group must consist of **three students**.

Although all group members are expected to understand the entire system, responsibilities should initially be distributed among the members.

## Student A: Core Data Management

Primary responsibilities:

- Resource data structures
- Dynamic storage
- Linked list operations
- Searching and sorting
- File loading and saving

## Student B: Request and Activity Management

Primary responsibilities:

- Queue implementation
- Stack implementation
- Priority queue or heap
- Request processing
- Activity scheduling

## Student C: Advanced Data Structures and Navigation

Primary responsibilities:

- Trees
- Hash tables
- Graph representation
- Graph traversal
- Path or connectivity analysis

### Important

The division of responsibilities does **not** mean that students may work independently without understanding the other components.

During demonstrations and assessments, **any group member may be asked questions about any part of the system**.

---

# 6. Functional Requirements

## Module 1: Resource Management

The system must manage a collection of campus resources.

Each resource should contain appropriate information such as:

- Resource ID
- Resource name
- Resource type
- Location
- Capacity
- Availability status

For example:

Resource ID: LAB101
Name: ICTLAB
Type: Computer Laboratory
Location: Building 32
Capacity: 40
Status: Available

The system must allow users to:

1. Add a resource.
2. Remove a resource.
3. Update resource information.
4. Display all resources.
5. Search for a resource.
6. Sort resources according to selected criteria.

---

# 7. Module 2: Dynamic Resource Storage

The project must demonstrate the use of a dynamic data structure.

Students may use a linked list to store resources or another component of the system.

The implementation must demonstrate appropriate operations such as:

- Insertion at the beginning.
- Insertion at the end.
- Insertion at a specified position.
- Deletion.
- Traversal.
- Searching.

Students should explain why a linked structure may be more appropriate than a fixed-size array in the selected part of their system.

### Minimum Requirement

At least one major part of the system must use a **linked list implementation designed or implemented by the students**.

Students should not simply use `std::list` without demonstrating an understanding of the underlying data structure.

---

# 8. Module 3: Request Management Using Queues

Users may submit requests for campus resources.

Examples include:

- Requesting a laboratory.
- Requesting a lecture room.
- Requesting equipment.
- Requesting access to a facility.

Normal requests should be processed according to the **First-In-First-Out (FIFO)** principle.

Each request should contain information such as:

- Request ID
- User information
- Requested resource
- Request time
- Request status

The system must demonstrate:

- Enqueueing requests.
- Dequeueing requests.
- Viewing pending requests.
- Processing requests.

Students must clearly demonstrate how the queue supports the FIFO processing model.

---

# 9. Module 4: Undo Operations Using a Stack

The system must provide at least one operation that demonstrates the use of a **stack**.

Possible examples include:

- Undoing the last resource modification.
- Undoing the last request cancellation.
- Tracking recently visited system screens.
- Reversing a sequence of operations.

For example:

Add Resource
↓
Update Resource
↓
Delete Resource
↓
UNDO
↓

Restore Deleted Resource

The system must demonstrate:

- Push
- Pop
- Peek or Top
- Empty stack detection

Students must explain why a stack is appropriate for the selected functionality.

---

# 10. Module 5: Priority Request Management

Not all requests have equal importance.

The system must support a priority-based request mechanism.

Possible priority categories include:

1. Emergency
2. High
3. Normal
4. Low

For example:

| Request | Priority |
|---|---|
| Emergency laboratory maintenance | 1 |
| Examination room request | 2 |
| Lecture room request | 3 |
| General meeting request | 4 |

Higher-priority requests should be processed before lower-priority requests.

This component should be implemented using either:

- A priority queue, or
- A heap.

The system must demonstrate:

- Insertion.
- Priority-based processing.
- Removal of the highest-priority request.

Students must explain the relationship between **heaps and priority queues**.

---

# 11. Module 6: Searching and Sorting

The system must provide mechanisms for searching and organising information.

## Searching

The system should support at least two searching approaches.

Possible examples include:

- Linear search.
- Binary search.

Students should compare the algorithms and discuss situations where each approach is appropriate.

For binary search, students must ensure that the relevant collection is sorted.

---

## Sorting

The system must implement and use at least **two sorting algorithms**.

Possible algorithms include:

- Selection sort
- Insertion sort
- Bubble sort
- Merge sort

- Quick sort
- Heap sort

Resources may be sorted according to:

- Resource ID
- Name
- Capacity
- Type

Requests may be sorted according to:

- Submission time
- Priority
- Status

### Requirement

At least one of the implemented sorting algorithms must have a significantly better average-case complexity than O(n2)O(n^2).

Students must compare the selected algorithms.

---

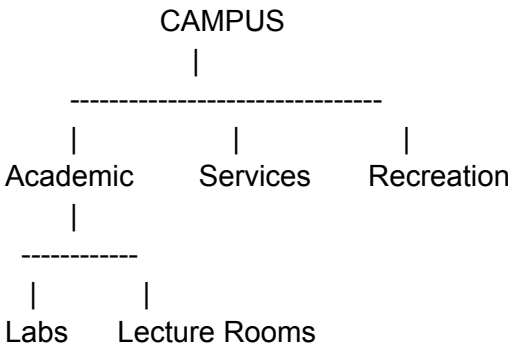
# 12. Module 7: Hierarchical Organisation Using Trees

The system must represent at least one type of hierarchical or searchable information using a **tree**.

Possible applications include:

- Organising resources according to categories.
- Organising campus facilities hierarchically.
- Maintaining resources in a Binary Search Tree.
- Managing users or records according to unique keys.

For example:




Alternatively, students may implement a **Binary Search Tree (BST)** where resources are organised according to their IDs.

The tree implementation must demonstrate appropriate operations such as:

- Insertion.
- Searching.
- Traversal.
- Deletion, where applicable.

Students should demonstrate at least two tree traversal methods.

Examples:

- Inorder traversal
- Preorder traversal
- Postorder traversal
- Level-order traversal

---

# 13. Module 8: Fast Resource Retrieval Using Hashing

The system must implement a mechanism for efficiently retrieving information using a **hash table**.

For example:

Resource ID → Resource Information

A user should be able to enter a resource identifier and retrieve the corresponding information efficiently.

Students must:

1. Define an appropriate hash function.
2. Implement a hash table.
3. Address collisions.

Possible collision handling techniques include:

- Separate chaining.
- Linear probing.
- Quadratic probing.
- Double hashing.

The group must justify its selected collision handling technique.

---

# 14. Module 9: Campus Connectivity Using Graphs

The campus contains different locations connected through pathways.

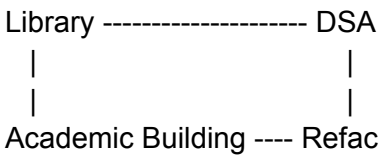
The system must model these locations using a graph.

Examples of vertices may include:

- Library
- Computer Science Building
- Administration Building
- Student Affairs
- Laboratories

Connections between locations represent edges.

For example:




The system must provide at least the following functionality:

1. Add a location.
2. Add a connection between locations.
3. Display connected locations.
4. Perform graph traversal.

Students must implement at least:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

Students should explain the differences between the two traversal approaches.

### Advanced Requirement

Groups should implement one additional graph-related feature, such as:

- Finding the shortest path.
- Determining whether a location is reachable.
- Detecting connected components.

---

# 15. Module 10: Data Persistence

The system should support saving and loading selected data.

For example:

resources.txt
requests.txt
locations.txt

The system should:

- Load data when the program starts.
- Save changes when the program terminates.

Students may use text files or another appropriate file format.

The focus of the project remains on **data structures and algorithms**, not database development.

---

# 16. Progressive Development Structure

The project should be developed in stages.

## Phase 1: Foundation

Implement:

- Basic menu system.
- Resource ADT.
- Dynamic storage.
- Basic CRUD operations.

### Main Concepts

- Classes
- Objects
- Arrays
- Linked lists
- Dynamic memory management

---

# Phase 2: Managing Requests

Implement:

- Regular request queue.
- Request processing.
- Stack-based functionality.

### Main Concepts

- Queue
- Stack
- Dynamic structures

---

# Phase 3: Efficient Organisation

Implement:

- Searching.
- Sorting.
- Tree-based organisation.
- Priority request management.

### Main Concepts

- Algorithm analysis
- Trees
- Heaps
- Priority queues

---

# Phase 4: Advanced Retrieval and Connectivity

Implement:

- Hash table.
- Graph.
- BFS.
- DFS.

### Main Concepts

- Hashing
- Graphs
- Traversal algorithms

---

## Phase 5: Integration and Analysis

Integrate all modules into a single system.

Students must:

- Test the system.
- Analyse selected algorithms.
- Compare alternative approaches.
- Prepare the final documentation.

---

# 17. Preventing Project Copying: Group Variations

**Each group will be assigned a unique project configuration.**

Groups will implement the same broad concepts but apply them to different scenarios.

Possible scenarios include:

### Group Type A: Campus Facilities

Manage:

- Lecture rooms
- Laboratories
- Buildings
- Equipment

### Group Type B: University Events

Manage:

- Events
- Venues
- Event requests
- Event priorities

## Group Type C: Library Resource Management

Manage:

- Books
- Study rooms
- Equipment
- Reservation requests

## Group Type D: Student Support Services

Manage:

- Appointments
- Service requests
- Support centres
- Emergency requests

## Group Type E: Campus Transport

Manage:

- Bus stops
- Routes
- Vehicles
- Transport requests

## Group Type F: Sports and Recreation

Manage:

- Sports facilities
- Equipment
- Bookings
- Activities

---

## Individual Group Parameters

In addition to receiving different scenarios, each group may receive unique parameters such as:

- Different resource attributes.
- Different sorting criteria.
- Different tree keys.
- Different hash table sizes.
- Different graph structures.
- Different priority rules.
- Different advanced features.

This means that while students learn from the same course concepts, **their systems will not be identical**.

---

# 18. Technical Requirements

The project must be implemented using **C++**.

Students are expected to demonstrate an understanding of data structure implementation.

Students should implement core data structures themselves rather than relying entirely on the C++ Standard Template Library.

The project should demonstrate appropriate use of:

- Classes.
- Encapsulation.
- Constructors and destructors.
- Dynamic memory management.
- Pointers and references where appropriate.
- Templates where appropriate.
- Separation of interface and implementation.

---

# 19. Code Organisation

A recommended project structure is:

```
CS3400_Mini_Project/
│
├── main.cpp
│
├── Resource.h
├── Resource.cpp
│
├── LinkedList.h
├── LinkedList.cpp
│
├── Queue.h
├── Queue.cpp
│
├── Stack.h
├── Stack.cpp
│
├── PriorityQueue.h
├── PriorityQueue.cpp
│
├── Tree.h
├── Tree.cpp
│
├── HashTable.h
├── HashTable.cpp
│
├── Graph.h
├── Graph.cpp
│
├── Sorting.h
├── Sorting.cpp
│
└── data/
    ├── resources.txt
    ├── requests.txt
    └── locations.txt
```

Groups may organise their code differently, provided that the final project is well structured and understandable.

---

# 20. Algorithm Analysis Requirement

Students must select at least **five significant operations** from their system and analyse their complexity.

Examples include:

| Operation | Data Structure/Algorithm | Time Complexity |
|---|---|---|
| Add to queue | Queue | O(1) |
| Search resource | Linear Search | O(n) |
| Search sorted resources | Binary Search | O(logn) |
| Hash table lookup | Hash Table | Average O(1) |
| Tree search | BST | Average O(logn) |
| BFS traversal | Graph | O(V+E) |

Students must discuss whether these complexities represent:

- Best case.
- Average case.
- Worst case.

They must also explain how their selected data structure affects the efficiency of the operation.

---

# 21. Testing Requirements

Each group must test:

### Normal Cases

The system behaves correctly with valid input.

### Boundary Cases

Examples:

- Empty queue.
- Empty stack.
- Empty tree.
- Empty graph.
- Full hash table where applicable.

### Invalid Cases

Examples:

- Searching for a non-existent resource.
- Removing a resource that does not exist.
- Invalid menu input.
- Duplicate identifiers.

### Large Data Cases

The system should be tested with a reasonably large dataset where applicable.

Students should demonstrate that their algorithms and data structures behave correctly under increasing data volumes.

---

# 22. Required Documentation

Each group must submit a concise technical report containing:

## 1. Introduction

- Description of the problem.
- Project objectives.
- Scope of the system.

## 2. System Design

- Overall system architecture.
- Class design.
- Description of major data structures.

## 3. Data Structure Selection

For each major data structure:

- What problem does it solve?
- Why was it selected?
- What alternatives were considered?

## 4. Algorithms

Describe the major algorithms used.

Include:

- Searching.
- Sorting.
- Tree operations.
- Graph traversal.

## 5. Complexity Analysis

Analyse selected operations.

## 6. Testing

Present selected test cases and results.

## 7. Challenges and Lessons Learned

Discuss:

- Technical challenges.
- Design challenges.
- Team collaboration.
- Important lessons.

---

# 23. Project Deliverables

Each group must submit:

### 1. Complete Source Code

All source and header files.

### 2. Technical Report

In PDF format.

### 3. README File

The README must explain:

- How to compile the program.
- How to run the program.
- Group members.
- Major features.

### 4. Sample Data

The project must include sufficient sample data to demonstrate the system.

### 5. Presentation and Demonstration

Each group will demonstrate its working system.

---

# 24. Demonstration Requirements

During the demonstration, the group must demonstrate:

1. Adding and managing records.
2. Linked list operations.
3. Queue operations.
4. Stack operations.
5. Priority processing.
6. Searching.
7. Sorting.
8. Tree operations.
9. Hash table retrieval.
10. Graph traversal.

The lecturer may request that a specific operation be demonstrated during the presentation.

---

# 25. Individual Accountability

Although this is a group project, every student must understand the complete system.

During the demonstration:

- Individual students may be asked to explain specific code.
- Individual students may be asked to modify or trace an algorithm.
- Students may be asked about time complexity.
- Students may be asked to justify a design decision.

A student who cannot demonstrate reasonable understanding of the project may receive a mark different from the overall group mark.

---

# 26. Assessment Rubric

| Criterion | Marks |
|---|---|
| Problem understanding and system design | 10 |
| Appropriate use of data structures | 20 |
| Algorithms, searching and sorting | 15 |
| Trees, hashing and priority management | 15 |
| Graph implementation and traversal | 10 |
| Code quality and organisation | 10 |
| Complexity analysis | 5 |
| Testing and robustness | 5 |
| Documentation | 5 |
| Demonstration and individual understanding | 5 |
| **Total** | **100** |

---

# 27. Important Academic Integrity Requirement

Groups may discuss general programming concepts with other students. However:

- Source code must be written by the members of the group.
- Groups must not copy another group's implementation.
- Groups must not submit code obtained from another group as their own.
- External libraries or code must be appropriately acknowledged where permitted.
- Every group member must be able to explain the submitted work.

Projects that are substantially similar may be investigated for academic misconduct.

---

# 28. Minimum System Menu

A possible menu structure is:

```
==============================================
CS3400 CAMPUS RESOURCE MANAGEMENT SYSTEM
==============================================

1. Resource Management
2. Search Resources
3. Sort Resources
4. Resource Requests
5. Priority Requests
6. Undo Operations
7. Tree Operations
8. Hash Table Lookup
9. Campus Navigation
10. System Analysis
0. Exit

Enter your choice:
```

---

# 29. Challenge Extensions

Groups that complete the minimum requirements early may implement additional features such as:

- Balanced trees.
- Dijkstra's shortest path algorithm.
- Multiple sorting algorithm comparisons.
- Performance measurement using different dataset sizes.
- Custom templates for data structures.
- Graph visualisation.
- Advanced collision resolution techniques.
- Additional undo/redo functionality.

These extensions should **not replace the core project requirements**.

---

# 30. Final Instruction to Students

This project is not simply an exercise in building a menu-driven C++ application. Its primary purpose is to demonstrate your understanding of how data structures and algorithms can be used to solve practical computing problems.

You are expected to make deliberate design decisions. For every major data structure or algorithm used, you should be able to explain:

- What problem it solves.
- Why it is appropriate.
- How it works.
- Its time and space complexity.
- What alternative approaches could have been used.

A working program without demonstrated understanding will not receive full credit.

---